

# Paraphrased Lab 02 Python Script Prompt

## Main Task

Create a complete and self-contained Python program named **lab02\_miranda.py**. The program must read and analyze the reservoir stage data from the Excel file **C:\Users\Lenovo\Downloads\Data 01.xlsx** and automatically produce one HTML dashboard called **lab02\_miranda.html**.

## 1. Data Import and Preparation

Read the Excel dataset using the fourth row as the header, which corresponds to a header index of 3. Clean and standardize the timestamp information, then convert the time values into elapsed hours beginning at **0.00 hours** and increasing by **0.25 hour** for each observation.

## 2. Numerical Derivatives

Use finite-difference methods to calculate the first derivative of stage with respect to time, **dh/dt**, and the second derivative, **d²h/dt²**. Use these results to determine the maximum inflow rate and report the exact time, in hours, when that maximum occurs.

## 3. Logistic Curve Fitting

Fit the measured reservoir stage data to a four-parameter logistic equation of the form:

$$h(t) = c + a / [1 + \exp(-k(t - t_{\text{mid}}))]$$

Use **scipy.optimize.curve\_fit** with the Levenberg–Marquardt method to estimate the four model parameters. Make sure the fitting process is handled safely so that invalid or missing numerical values do not cause the program to fail.

## 4. JSON and Chart Data Handling

Before placing derivative results and other numerical arrays into the HTML/JavaScript section, convert **NaN** values into JSON-compatible **null** values. Ensure that all arrays have compatible lengths and shapes so the embedded Chart.js code does not produce **TypeError**, **dimension mismatch**, or **invalid JSON** errors.

## 5. Statistical Analysis

Evaluate the quality of the logistic model using the following statistical measures: **SSE** (sum of squared errors), **SST** (total sum of squares), **R²** (coefficient of determination), and **s** (standard error of the estimate).

For each fitted parameter, calculate its standard error, t-statistic, and two-tailed p-value. Present these results clearly in the HTML dashboard.

## 6. Numerical Integration

Integrate the fitted logistic function using **scipy.integrate.quad**. Then independently calculate the same integral using **np.trapezoid** and display both results so they can be compared as a numerical cross-check.

## 7. Dashboard Design

Build the output as a clean, modern, light-themed technical dashboard appropriate for a formal engineering or academic submission. Use a pure white background (**#ffffff**), light gray cards (**#f8f9fc**), dark navy text (**#0f172a**), and subtle slate borders (**#e2e8f0**).

## 8. Executive Header and KPI Section

Place a formal document title and relevant metadata at the top of the dashboard. Below the header, create a four-column KPI/highlight area that presents the most important summary results in a clear and professional format.

## 9. Interactive Tabs

Add an interactive navigation bar with exactly three main tabs:

- **Tab 1:** Numerical Derivatives & Kinematics
- **Tab 2:** Logistic Model & Curve Fitting
- **Tab 3:** Statistical Diagnostics & Volumetric Integration

## 10. Charts and Layout

Organize the dashboard using side-by-side card layouts where appropriate. Each section should have a clear heading and short engineering notes explaining what the displayed results mean. Use interactive Chart.js graphs with a professional blue, teal, and orange palette based on **#0284c7**, **#0d9488**, and **#d97706**.

## 11. Parameter Results Table

Create a formatted parameter table for the logistic model. Include the estimated values and their statistical information. Add subtle hover effects and color-coded status labels to make important results easy to identify without making the dashboard look overly decorative.

## 12. Final Output Requirements

The final response should contain only the executable Python source code for the requested program. The script must be standalone, robust, and ready to run on the specified Windows machine. It must generate the complete HTML dashboard automatically, with all numerical results and Chart.js data embedded correctly.

**Expected files:** lab02\_miranda.py and the generated lab02\_miranda.html